

LLMs in Requirements Engineering

From stakeholder intent to structured and verifiable software specifications

References

1. C. Arora, J. Grundy, M. Abdelrazek, “Advancing Requirements Engineering through Generative AI: Assessing the Role of LLMs,” 2023.
2. K. Ronanki, C. Berger, J. Horkoff, “Investigating ChatGPT’s Potential to Assist in Requirements Elicitation Processes,” 2023.
3. L. Ma, S. Liu, Y. Li, X. Xie, L. Bu, “SpecGen: Automated Generation of Formal Program Specifications via Large Language Models,” 2025.

Key concepts used in this lecture:

Requirements elicitation • requirements specification • EARS • user stories • functional/NFR classification • prompt engineering • formal specifications • JML • verification • mutation-based generation • human-in-the-loop RE

What should you learn?

1. Reframe RE

See requirements engineering as a core engineering activity—not paperwork before coding.

2. Understand LLM roles

See how LLMs can elicit, structure, refine, classify and generate specifications.

3. Understand limits

Learn why human expertise, verification and stakeholder validation remain essential.

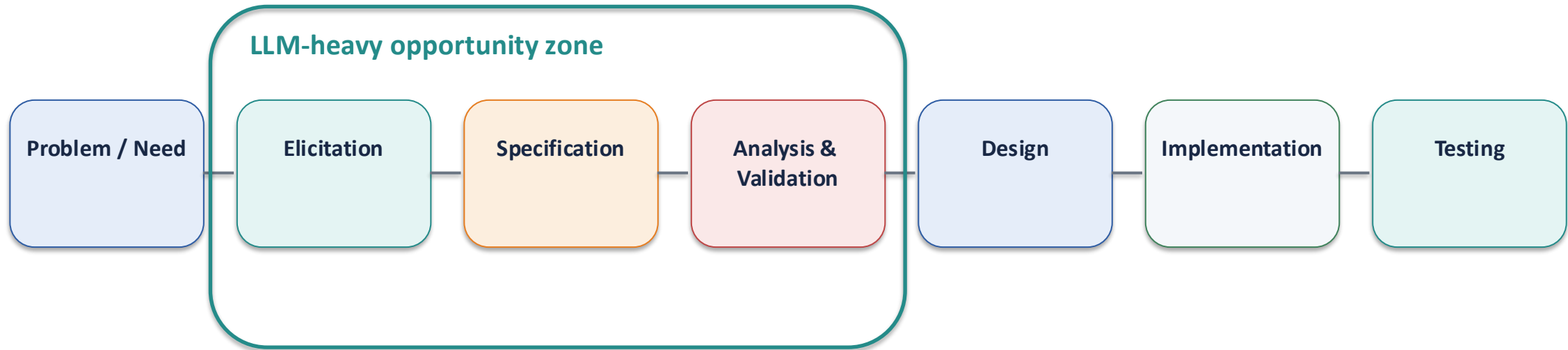
4. Connect to SDLC

Trace the path from natural-language intent → requirements → formal specifications → verification.

5. Think like an engineer

Use LLMs as collaborators inside a controlled workflow—not as autonomous requirement authorities.

Where do LLMs fit in the SDLC?



Elicit stakeholder needs

Convert informal language into structured requirements

Identify ambiguity, gaps and conflicts

Generate formal specifications and testable constraints

Three papers, three levels of the problem

Paper 1 — RE vision

Arora, Grundy & Abdelrazek

“Advancing Requirements Engineering through Generative AI: Assessing the Role of LLMs”

LLMs across:

- elicitation
- specification
- analysis
- validation

Key idea: LLMs can become RE assistants/agents, but humans remain central.

Paper 2 — Elicitation evidence

Ronanki, Berger & Horkoff

“Investigating ChatGPT’s Potential to Assist in Requirements Elicitation Processes”

Controlled comparison of ChatGPT-generated and human-generated requirements for Trustworthy AI.

Key idea: generated requirements can be strong on several quality attributes, but shortcomings remain.

Paper 3 — Formal specification

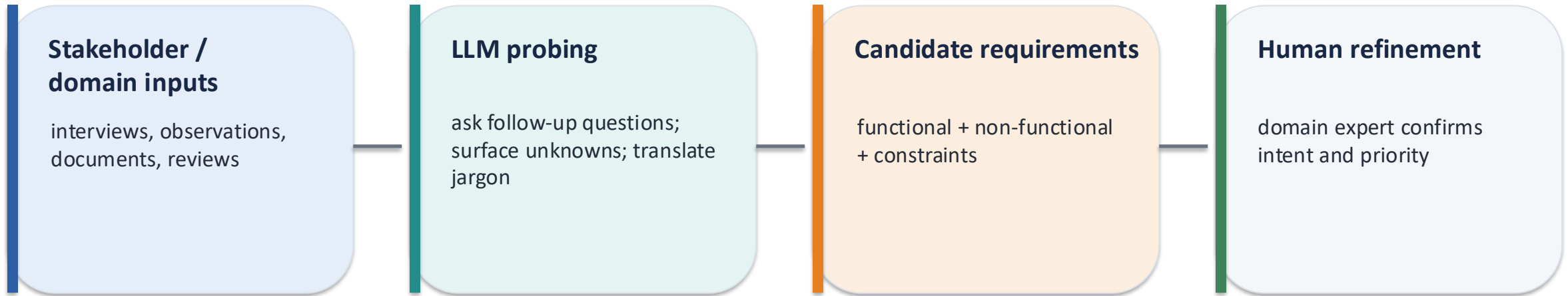
Ma et al.

“SpecGen: Automated Generation of Formal Program Specifications via Large Language Models”

LLM + conversation + verifier + mutation.

Key idea: do not trust raw LLM output; close the loop with machine verification.

LLMs can turn elicitation into an interactive dialogue

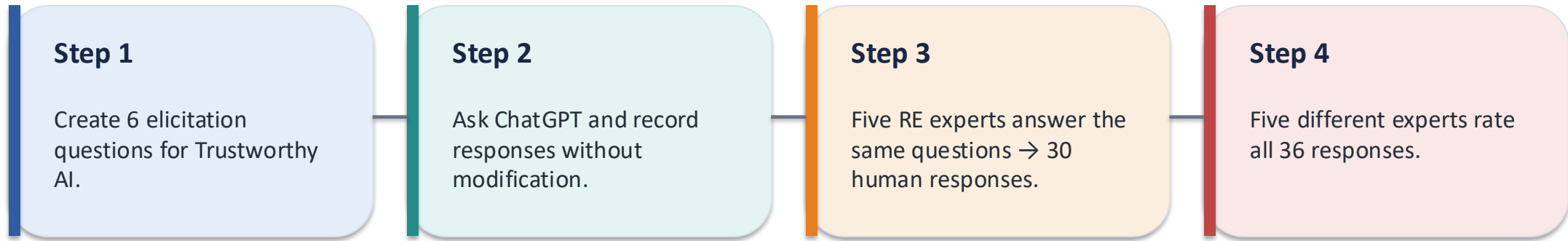


The first paper proposes LLMs for domain analysis, stakeholder communication, gap discovery, multilingual support and human-centric elicitation.

A concrete example uses an LLM to act as a persona and generate preliminary requirements from the user's perspective.

The critical boundary: LLM output is complementary to stakeholder and domain expertise.

Paper 2: How was ChatGPT actually evaluated?



Seven quality attributes were scored from 0–10: Abstraction, Atomicity, Consistency, Correctness, Unambiguity, Understandability and Feasibility.

Evaluators were not told which responses were generated by ChatGPT, reducing direct evaluator bias.

The study is a controlled exploratory study—not evidence that LLMs can replace RE experts.

Paper 2: What did ChatGPT do well?

Strong attributes

The abstract reports high performance on:

- Abstraction
- Atomicity
- Consistency
- Correctness
- Understandability

What this means

LLMs can produce requirements that are structured, readable and reasonably coherent—even in a niche topic such as Trustworthy AI.

But not “done”

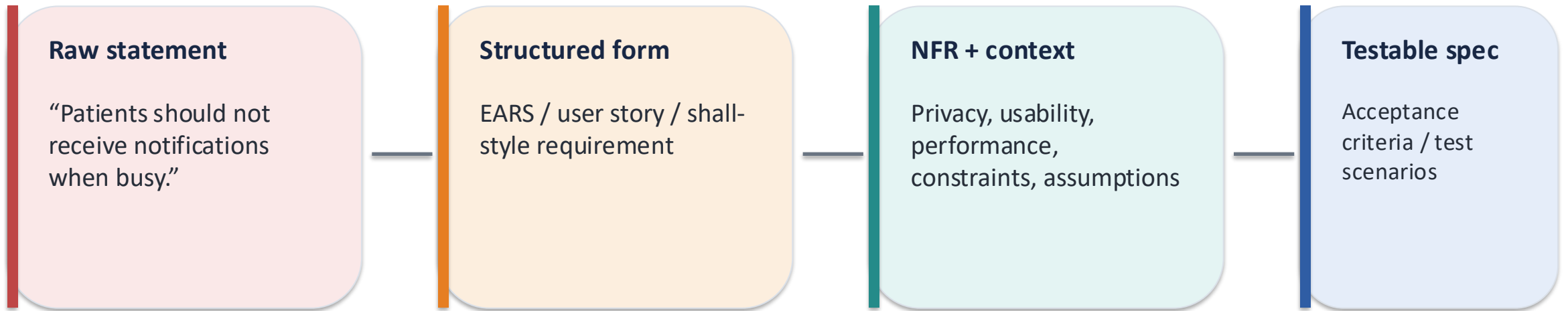
The paper explicitly investigates shortcomings and compares quality against human experts; the study does not establish autonomous requirements engineering.

Trustworthy AI itself illustrates why elicitation must cover accuracy, robustness, fairness, transparency, privacy/security and human oversight.

If the LLM gives a plausible requirement, what evidence tells us that the requirement is actually necessary for this product?

Engineering lesson: quality of wording $\neq$ correctness of business intent.

Specification stage: from “what people said” to “what the system shall do”



Arora et al. explicitly position LLMs as tools for converting informal natural-language requirements into EARS, user stories and structured requirements documents.

LLMs can classify functional vs. non-functional requirements, generate glossaries/rationales/personas and perform initial compliance cross-checks.

The danger: a beautifully formatted requirement can still be ambiguous, over-specified or missing important NFRs.

Prompt engineering becomes part of requirements engineering

Weak prompt

“Write requirements for a food delivery app.”

Problem: underspecified context, audience, scope and output.

Better prompt

Context + role/persona + task + constraints + examples + expected output format.

Engineering prompt

Require IDs, rationale, assumptions, NFRs, acceptance criteria, conflicts and “unknowns”.

Arora et al. describe prompt patterns such as output customization and a consistent Context–Task–Expected Output structure.

It discusses zero-shot, few-shot, chain-of-thought and tree-of-thought prompting as techniques whose combinations need empirical evaluation for RE.

The key shift: the prompt is no longer merely a query—it becomes a specification of the specification task.

Paper 3: LLMs can generate formal program specifications

What is a formal specification?

A precise description of program behavior: preconditions, postconditions, loop invariants and assertions.

Why is it hard?

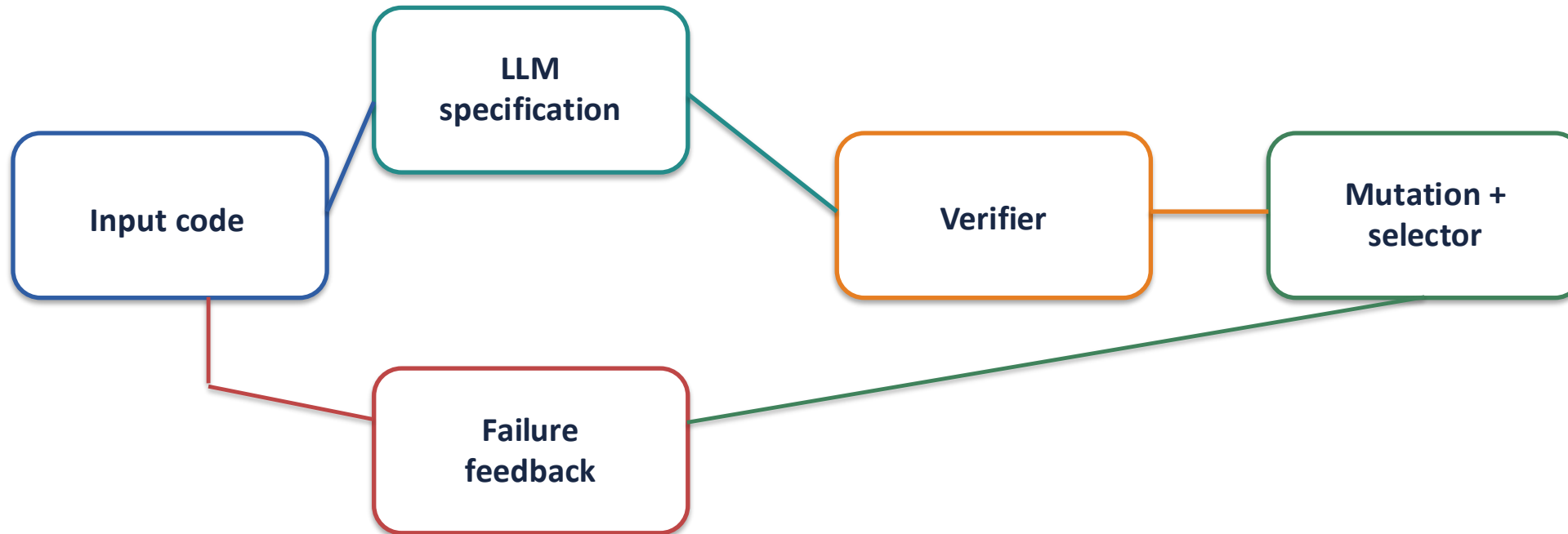
Manually writing formal specifications is time-consuming and difficult for complex real-world programs.

Why LLMs?

LLMs can understand code semantics beyond the narrow templates used by conventional tools.

- SpecGen targets Java programs and JML-style specifications.
 - For example, in Java, the specifications can be expressed in Java Modeling Language (i.e., JML) [23] where `requires` statements denote the pre-conditions of a function, `ensures` statements represent the post-conditions of a function, and `maintaining` statements specify the loop invariants.
- It focuses on specifications that describe actual program behavior and can support verification, testing and quality assurance.
- This is a different level of SDLC automation: the LLM is no longer just writing prose requirements—it is generating machine-checkable constraints.

SpecGen: the important idea is the closed feedback loop



Phase 1: conversational generation using few-shot examples; verifier failures become feedback for the next LLM attempt.

Phase 2: when generation fails, four mutation types generate candidate variants; a heuristic selector prioritizes promising candidates for verification.

Engineering principle: generation proposes; verification disposes.

Evidence: SpecGen improves verifiable specification generation

SpecGen



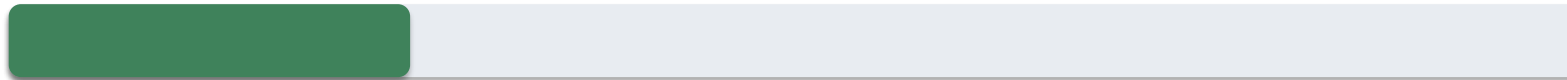
279/385

AutoSpec



247/385

Houdini



98/385

SpecGen successfully generated verifiable specifications for 279 of 385 programs ($\approx 72.5\%$).

What should we delegate to an LLM—and what should we not?

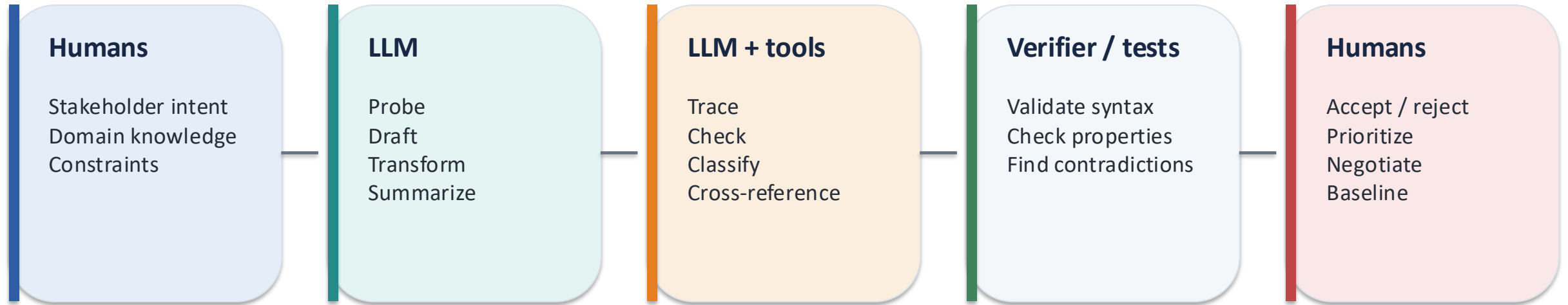
Good candidates for LLM assistance

- ✓ Transform and normalize natural-language requirements
- ✓ Generate alternative formulations
- ✓ Produce user stories / EARS / shall-style drafts
- ✓ Classify functional vs. non-functional requirements
- ✓ Extract assumptions, glossary terms and candidate constraints
- ✓ Ask probing questions and identify ambiguity/gaps
- ✓ Summarize stakeholder feedback
- ✓ Generate candidate formal specifications
- ✓ Iterate using machine-generated feedback

Keep humans accountable for

- ✗ Deciding what the customer actually needs
- ✗ Domain-specific trade-offs and feasibility
- ✗ Safety-critical or legally consequential decisions
- ✗ Business priorities and conflicting stakeholder goals
- ✗ Determining whether an NFR is truly sufficient
- ✗ Approving compliance interpretations
- ✗ Accepting a requirement merely because it “sounds right”

A practical Human–LLM Requirements Engineering workflow



The strongest pattern across the papers is not “LLM replaces engineer”; it is “LLM sits inside an iterative engineering process.”

The SpecGen paper makes this especially explicit: LLM output is coupled to verification and iterative correction.

What changes in the agentic era?

Old workflow

Stakeholder → analyst → document
→ review → implementation

LLM-assisted workflow

Stakeholder ↔ LLM analyst ↔
structured requirements ↔ human
review

Agentic workflow

Multiple specialized agents + tools +
traceability + verification + human
approval

A requirements agent could act as interviewer, requirements analyst, specification writer, critic and traceability assistant.

A second agent/tool can challenge the first: search regulations, identify missing NFRs (Non-functional requirements), detect conflicts, or generate acceptance tests.

But the system needs governance: provenance, versioning, access control, privacy, evaluation, audit trails and explicit human approval gates.

Takeway: software engineering shifts from “write everything manually” toward “design and control a reliable socio-technical pipeline.”

In-class exercise: build an LLM-assisted RE pipeline

Scenario

Build a campus food-delivery app for students, restaurants and delivery partners.

Task A — Elicit

Write a prompt that makes an LLM interview 3 personas and identify requirements + unknowns.

Task B — Specify

Ask the LLM to. convert the results into functional requirements, NFRs, user stories and acceptance criteria

Task C — Attack the output: find at least 5 ambiguities, missing requirements or unjustified assumptions.

Five takeaways

1

Requirements are engineering artifacts

Bad requirements create downstream cost, delay and quality problems.

2

LLMs are strong language collaborators

They can elicit, transform, classify, summarize and generate candidate specifications.

3

Formatting is not validation

A polished requirement can still be wrong, incomplete or irrelevant.

4

Verification closes the loop

SpecGen demonstrates the value of coupling LLM generation with a verifier and iterative correction.

5

Humans remain accountable

Domain knowledge, negotiation, prioritization, ethics and final acceptance cannot simply be delegated.

References

1. C. Arora, J. Grundy, M. Abdelrazek, “Advancing Requirements Engineering through Generative AI: Assessing the Role of LLMs,” 2023.
 2. K. Ronanki, C. Berger, J. Horkoff, “Investigating ChatGPT’s Potential to Assist in Requirements Elicitation Processes,” 2023.
 3. L. Ma, S. Liu, Y. Li, X. Xie, L. Bu, “SpecGen: Automated Generation of Formal Program Specifications via Large Language Models,” 2025.
-